

HDOC - DAY 4

C Programming Assignment

Multiplication of Two Integers Without Using the * Operator

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 29-08-26

Objective

The aim of this assignment is to multiply two integers without using the multiplication operator. Instead, the product is found using repeated addition. The solution should work for positive numbers, negative numbers and zero, and it should avoid doing extra additions when a shorter calculation is possible.

1. Understanding the Given Algorithm

The given method starts the product from zero and repeatedly adds the first number. The basic idea is easy to understand, but after trying it with different inputs, a few problems become clear.

Given Algorithm

```
Define num1, num2, prod and index as integers.  
Read num1 and num2.  
Set prod = 0 and index = 1.  
Repeat while index <= num2:  
    prod = prod + num1  
    index = index + 1  
Print prod.  
Stop.
```

Task 1 - Mistakes in the Given Algorithm

After checking the algorithm with the test cases, I found that it works only in some situations. The biggest problem occurs when num2 is negative. Since index starts from 1 and only increases, a condition such as $1 \leq -5$ is already false, so the loop never starts and the product stays 0.

- It does not properly handle a negative value of num2.
- There is no clear step for deciding whether the final answer should be positive or negative.
- Zero is not handled directly, even though the answer is immediately known when either input is zero.
- The loop always depends on num2 instead of choosing the smaller magnitude.
- This can cause unnecessary additions. For example, 3×12 can be found by adding 12 only three times instead of adding 3 twelve times.
- Because of these issues, the original logic cannot pass all the required test cases.

Task 2 - Corrected Algorithm

To make the solution reliable, I first deal with zero and the sign of the answer. Then I convert both numbers to positive magnitudes. Finally, I use the smaller magnitude as the loop count and keep adding the larger one.

Step 1: Start the algorithm.

Step 2: Read num1 and num2.

Step 3: If num1 is 0 or num2 is 0, print 0 and stop.

Step 4: Set sign = 1.

Step 5: If num1 is negative, change num1 to -num1 and reverse the value of sign.

Step 6: If num2 is negative, change num2 to -num2 and reverse the value of sign.

Step 7: Compare the positive values. Store the smaller value in count and the larger value in addend.

Step 8: Set product = 0 and index = 1.

Step 9: While index <= count, add addend to product and increase index by 1.

Step 10: If sign = -1, make product negative.

Step 11: Print product.

Step 12: Stop.

Why this is better

The loop now works only with positive magnitudes, so negative inputs do not disturb the repetition condition. The sign is handled separately and applied at the end. Also, using the smaller value as count keeps the number of repeated additions as low as possible for this method.

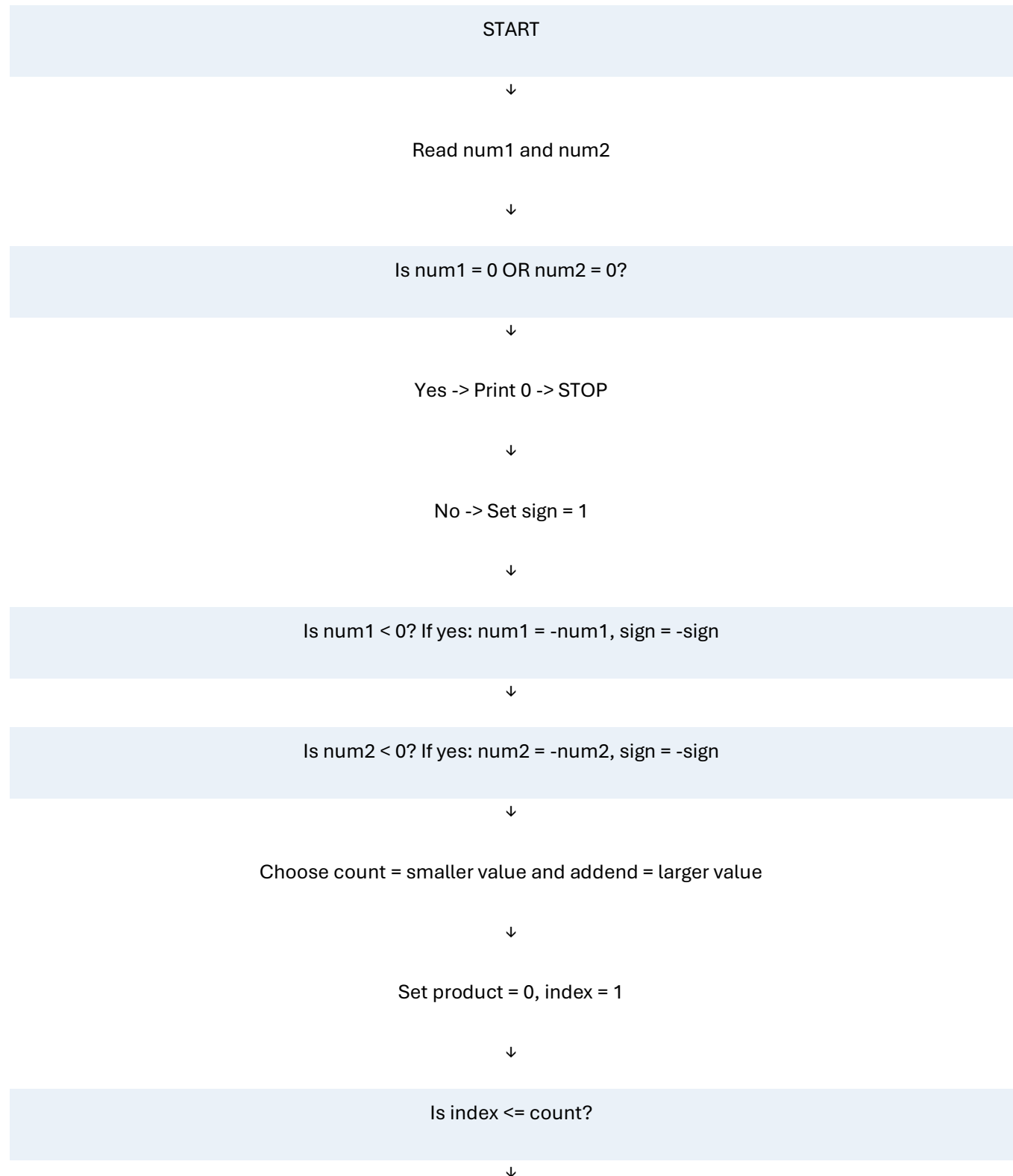
Task 3 - Mistakes in the Given Flowchart

The original flowchart follows the same incomplete logic as the given algorithm. It does not show what should happen for zero or negative inputs, and it does not compare the magnitudes before starting the loop. Because of this, it cannot represent a complete solution for all the test cases.

- No separate zero check.
- No proper sign handling.
- Negative values are not converted to positive magnitudes before looping.
- The smaller magnitude is not selected as the loop counter.
- The flowchart therefore gives wrong or inefficient results for several input combinations.

Task 4 - Correct Flowchart

A simple way to represent the corrected logic is shown below. The decisions are arranged so that special cases are handled before the repeated-addition loop.



Yes -> $\text{product} = \text{product} + \text{addend}$; $\text{index} = \text{index} + 1$; return to condition



No -> If $\text{sign} = -1$, $\text{product} = -\text{product}$



Print product



STOP

5. C Program

The following program follows the corrected algorithm. I have kept the logic simple so that each part of the solution can be matched with the algorithm above.

```
#include <stdio.h>

int multiply(int num1, int num2)
{
    int product = 0, count, addend, sign = 1;
    int index = 1;

    if (num1 == 0 || num2 == 0)
        return 0;

    if (num1 < 0) {
        num1 = -num1;
        sign = -sign;
    }

    if (num2 < 0) {
        num2 = -num2;
        sign = -sign;
    }

    if (num1 < num2) {
        count = num1;
        addend = num2;
    } else {
        count = num2;
        addend = num1;
    }

    while (index <= count) {
        product = product + addend;
        index++;
    }

    if (sign == -1)
        product = -product;

    return product;
}

int main()
{
    int num1, num2;
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);
    printf("Product = %d\n", multiply(num1, num2));
    return 0;
}
```

How the program works

If either input is zero, the function returns zero immediately. For a negative input, its magnitude is made positive and the sign variable is reversed. The smaller magnitude becomes count, so the loop performs only the required number of additions. At the end, the stored sign is applied to the product.

Task 5 - Verification Using Test Cases

I checked the corrected logic using all 13 test cases given in the assignment. The product is correct in every case, and the number of repeated additions matches the required minimum.

TC	Input	Expected	Actual	Min. Additions	Result
1	12, 3	36	36	3	Pass
2	3, 12	36	36	3	Pass
3	-6, 5	-30	-30	5	Pass
4	6, -5	-30	-30	5	Pass
5	-9, -4	36	36	4	Pass
6	-4, -9	36	36	4	Pass
7	0, 8	0	0	0	Pass
8	8, 0	0	0	0	Pass
9	0, -7	0	0	0	Pass
10	-7, 0	0	0	0	Pass
11	0, 0	0	0	0	Pass
12	1, -7	-7	-7	1	Pass
13	-1, -9	9	9	1	Pass

Conclusion

The original repeated-addition idea is useful, but it needs a few changes to work for every integer input. The corrected solution handles zero first, deals with the sign separately and uses the smaller magnitude as the loop counter. This makes the method work correctly for positive-positive, positive-negative, negative-positive and negative-negative cases while avoiding unnecessary additions.

All 13 given test cases pass with the required minimum number of additions.